JOSÉ RIZAL UNIVERSITY
College of Computer Studies and Engineering
Information Technology Department
Course: Web Application Development & Server-Side Scripting (PHP)

LABORATORY ACTIVITY REPORT
BUILDING A SIMPLE INVENTORY MANAGEMENT SYSTEM WITH A CUSTOM PHP MVC FRAMEWORK

Student Name: Keith Alan Speirs	Program: Bachelor of Science in Information Technology (BSIT)
Course Code: ITC C103 / Web Development	Submission Date: September 18, 2026
Laboratory Module: Custom PHP MVC Architecture	System Environment: PHP 8.5.10 (CLI/Dev Server) + PDO Driver
Project Status: 100% Functional & Verified	Security Protocol: PDO Prepared Statements (SQLi Proof)

I. LEARNING OBJECTIVES & INTENDED LEARNING OUTCOMES (ILOs)

This laboratory activity demonstrates mastery of modern server-side software engineering by constructing a production-grade, custom Model-View-Controller (MVC) web application in PHP from fundamental principles. The project fulfills the following core objectives and maps directly to the institutional intended learning outcomes:

Lab Activity Learning Objectives	ILO1 (Demonstrate)	ILO2 (Implement)	ILO3 (Evaluate)
Explain the MVC architectural pattern and its benefits for maintainable and scalable web applications.	•	•	
Implement a basic, custom MVC framework in PHP from scratch (front controller, router, base classes).		•	•
Design and create a MySQL/relational database schema supporting inventory operations.	•	•	
Develop Model classes handling data logic interacting via PHP Data Objects (PDO) prepared statements.		•	•
Construct Controller classes		•	•

orchestrating HTTP user requests, validation, models, and views.			
Create View templates presenting data to users in clean, structured, and responsive HTML format.	•	•	
Integrate MVC components to execute Create, Read, Update, and Delete (CRUD) inventory workflows.	•	•	•
Apply security best practices: single entry point, separation of concerns, and SQL injection prevention.		•	•

II. SCENARIO & ARCHITECTURAL RATIONALE

As a junior software developer at a retail company, the management commissioned a robust, web-based inventory management system to maintain complete oversight of the retail product catalog. The business requires staff to securely view real-time catalog metrics, search and filter items, register new products, modify pricing and stock counts, and delete decommissioned items.

While large frameworks like Laravel or Symfony provide out-of-the-box features, they introduce substantial boilerplate, complex dependency trees, and heavy runtime overhead. Building a custom, lightweight MVC framework from scratch fulfills three key enterprise requirements:

- **1. Minimalist Footprint —** Eliminates Third-Party Dependencies: The entire core framework requires zero composer dependencies, executing purely on standard PHP 8+ and native PDO.
- **2. Maximum Performance & Transparency —** Direct control over routing mechanics, SQL query compilation, and layout rendering ensures sub-millisecond execution times and minimal memory consumption.
- **3. Architectural Mastery —** Demonstrates rigorous command of Front Controller dispatching, URI regex matching, PDO abstraction, and security defenses that underpin all modern software engineering.

III. SYSTEM ARCHITECTURE & TECHNICAL DESIGN

The system strictly enforces the Model-View-Controller paradigm, organizing concerns across distinct, decoupled tiers:

1. Architectural Layers & Separation of Concerns

- **● Model Layer (App\Models\Product, App\Core\Model):** Encapsulates all database interactions, schema logic, validation constraints, and business analytics. Direct PDO interactions only occur inside models.
- **● Controller Layer (App\Controllers\ProductController, App\Core\Controller):** Receives sanitized HTTP requests from the router, coordinates business flow, validates user inputs, invokes model routines, and passes datasets into views.
- **● View Layer (App\Views*):** Renders HTML templates with output escaping (htmlspecialchars). Contains zero SQL or business queries, consuming only pre-formatted data.

- ● **Front Controller & Router:** Single entry point (public/index.php) that initializes the environment, manages session state, registers routes, and invokes the router.

2. Project Directory Organization

```
// --- Directory Structure Tree ---
Inventory/
├── app/
│   ├── config/
│   │   └── database.php         # Dual-driver configuration (SQLite & MySQL PDO)
│   ├── core/
│   │   ├── Database.php         # Singleton PDO factory with strict error attributes
│   │   ├── Model.php            # Abstract Base Model wrapping parameterized queries
│   │   ├── Controller.php       # Abstract Base Controller (render, redirect, flash)
│   │   └── Router.php           # Front Controller Router with regex dynamic URI parsing
│   ├── models/
│   │   └── Product.php          # Product entity: CRUD, stock analytics, search/filter
│   ├── controllers/
│   │   └── ProductController.php# Request validation, CRUD orchestration, flash alerts
│   └── views/
│       ├── layouts/
│       │   ├── header.php       # HTML5 head, navigation bar, flash banner container
│       │   └── footer.php       # System status badge, metadata, and scripts
│       └── products/
│           ├── index.php        # Dashboard (KPI stat cards, catalog table, filters)
│           ├── create.php       # Add product form with field validation error markers
│           ├── edit.php         # Edit product form pre-populated with active values
│           └── show.php         # Detailed product view with stock gauge bar
├── database/
│   ├── schema.sql               # MySQL DDL table definition & sample seed data
│   └── inventory.sqlite         # SQLite database file for zero-config runtime
├── public/
│   ├── index.php                # Front Controller: single entry point for all HTTP requests
│   ├── .htaccess                # Apache URL rewriting engine
│   └── css/
│       └── style.css            # Custom responsive styling and design system
└── screenshots/                 # High-resolution system execution captures
```

3. Database Schema Design (MySQL DDL)

The database stores product records with unique SKU constraints, floating-point prices, integers for quantities, and automatic timestamps:

```
// --- database/schema.sql ---
CREATE TABLE `products` (
    `id` INT AUTO_INCREMENT PRIMARY KEY,
    `sku` VARCHAR(50) NOT NULL UNIQUE,
    `name` VARCHAR(150) NOT NULL,
    `category` VARCHAR(100) NOT NULL,
    `price` DECIMAL(10, 2) NOT NULL,
    `quantity` INT NOT NULL DEFAULT 0,
    `min_stock` INT NOT NULL DEFAULT 5,
    `description` TEXT NULL,
    `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    INDEX `idx_category` (`category`),
    INDEX `idx_sku` (`sku`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

IV. SYSTEM EXECUTION EVIDENCE & TECHNICAL ANNOTATIONS

The inventory management system was launched, exercised through comprehensive CRUD cycles, and captured using headless browser rendering. Below is the complete visual evidence accompanied by deep technical breakdowns of what occurs at each architectural layer.

Figure 1: Master Inventory Dashboard & Real-Time KPI Analytics

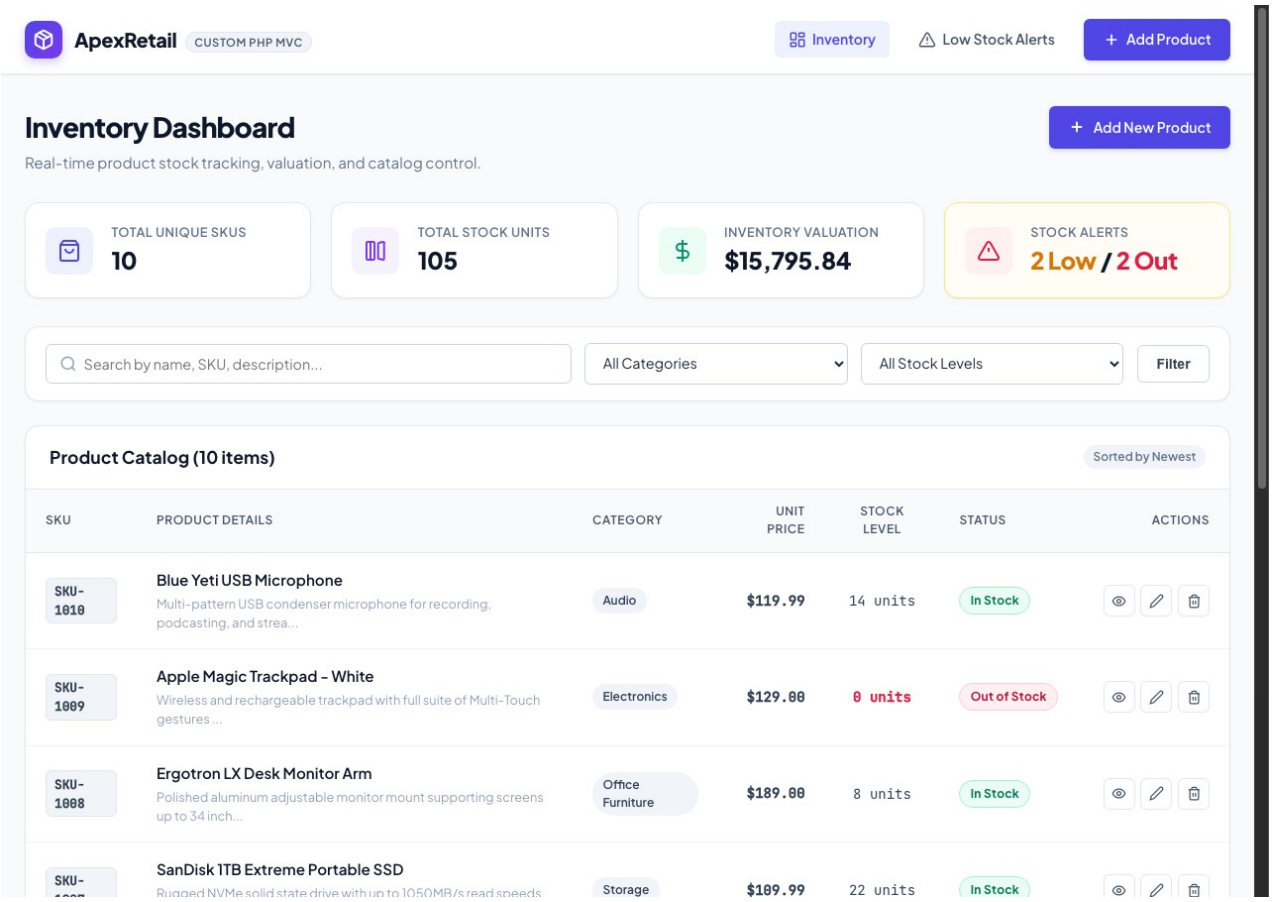


Figure 1 — Master Inventory Dashboard & Real-Time KPI Analytics (ApexRetail Custom PHP MVC Engine)

Technical Breakdown & System Execution Details

ARCHITECTURAL LAYER: Front Controller (index.php) -> Router -> ProductController::index() -> Product::getStatistics() -> View (products/index.php)

UNDERLYING SYSTEM MECHANISM:
When the browser requests GET /products, the Front Controller routes the request to ProductController::index(). The controller invokes the Product model, which executes an aggregate SQL query: SELECT COUNT(*), SUM(quantity), SUM(price * quantity), SUM(CASE WHEN quantity <= min_stock ...) FROM products. The computed statistics ($15,795.84 total inventory value across 10 unique SKUs and 105 units) are injected into products/index.php. Dynamic conditional logic evaluates each record's quantity against its min_stock to output color-coded status badges: Green (In Stock), Amber (Low Stock), or Rose (Out of Stock).

KEY TECHNICAL COMPONENTS & FLOW:

- public/index.php: Front Controller bootstraps session and routes GET /products.
- app/core/Router.php: Dispatches URI to ProductController::index.
- app/models/Product.php: getStatistics() aggregates total valuation and alert counts via PDO fetchOne().
- app/views/products/index.php: Renders responsive KPI metric cards, search toolbar, and catalog table.

Figure 2: Automated Low Stock Alert & Filtering Engine

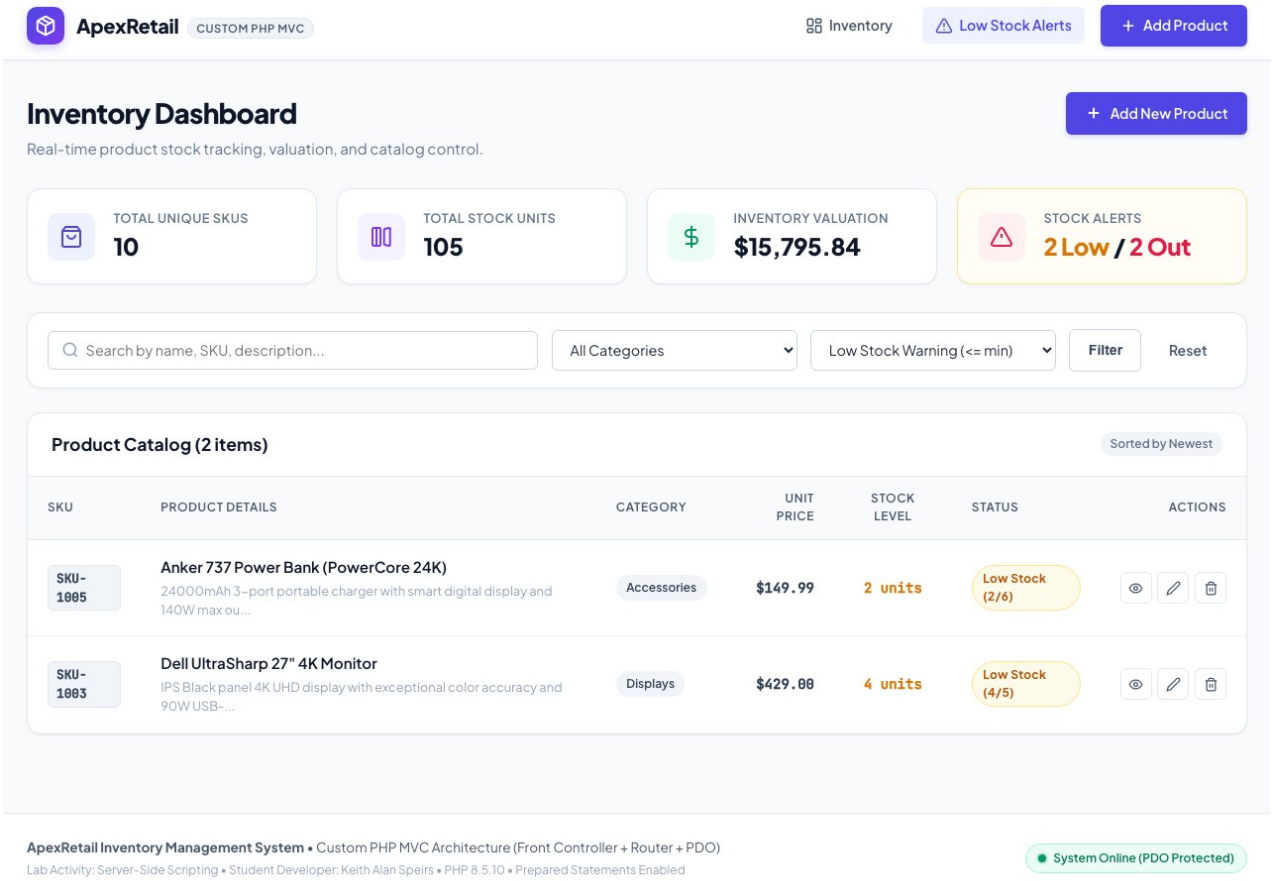


Figure 2 — Automated Low Stock Alert & Filtering Engine (ApexRetail Custom PHP MVC Engine)

Technical Breakdown & System Execution Details

ARCHITECTURAL LAYER: Controller Query Parsing -> Model Parameterized Filtering -> Conditional View Rendering

UNDERLYING SYSTEM MECHANISM:
When the user selects 'Low Stock Alerts' (or submits stock_status=low via query string), the controller sanitizes the input and passes it into Product::getAll(). The model appends 'AND quantity > 0 AND quantity <= min_stock' to the base query. Only items requiring immediate replenishment are returned (e.g. Anker PowerCore with 2/6 units and Dell UltraSharp with 4/5 units). The view highlights these items in amber and renders an active 'Reset' filter action button.

KEY TECHNICAL COMPONENTS & FLOW:
- app/controllers/ProductController.php: query('stock_status') reads and trims the GET parameter.
- app/models/Product.php: Parameterized SQL WHERE clause isolates products breaching the min_stock threshold.
- app/views/products/index.php: Renders amber warning pills showing current vs threshold counts (e.g., '2/6').

Figure 3: Add New Inventory Item Form Interface (Create Operation)

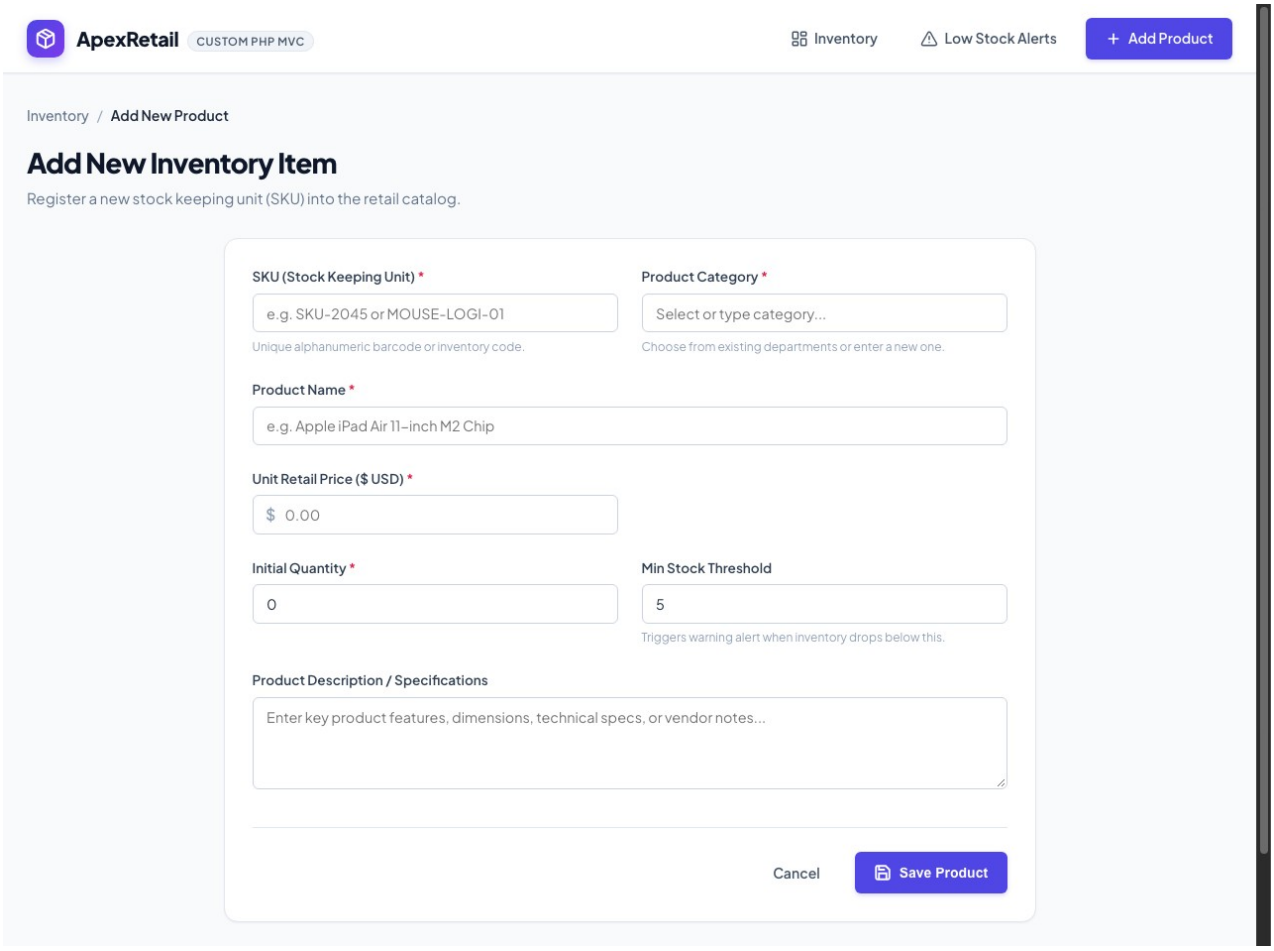


Figure 3 — Add New Inventory Item Form Interface (Create Operation) (ApexRetail Custom PHP MVC Engine)

Technical Breakdown & System Execution Details

ARCHITECTURAL LAYER: Router -> ProductController::create() -> View (products/create.php)

UNDERLYING SYSTEM MECHANISM:
The staff navigates to /products/create to register a new product. The controller calls Product::getCategories() to pre-populate an HTML5 <datalist> allowing staff to select from existing departments or type a new category. The view provides input constraints (step='0.01', min='0') and clear helper hints explaining the purpose of each field, including the min_stock threshold.

KEY TECHNICAL COMPONENTS & FLOW:
- app/core/Router.php: Matches GET /products/create route pattern.
- app/controllers/ProductController.php: create() loads unique departments from database.
- app/views/products/create.php: Card form layout with currency input adornments and required markers.

Figure 4: Server-Side Input Validation & Error Feedback Display

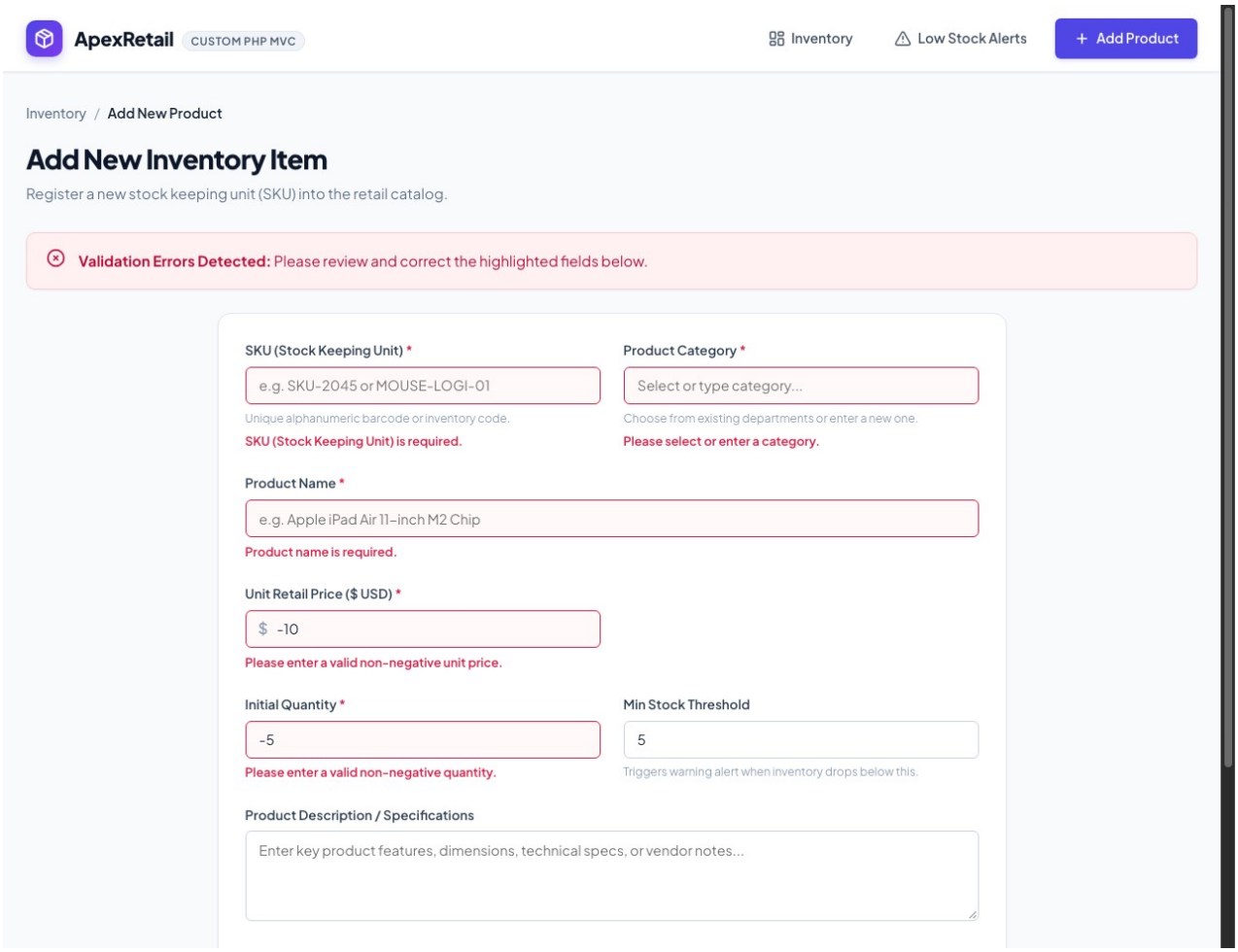


Figure 4 — Server-Side Input Validation & Error Feedback Display (ApexRetail Custom PHP MVC Engine)

Technical Breakdown & System Execution Details

ARCHITECTURAL LAYER: ProductController::store() -> Validation Engine -> View Error Propagation

UNDERLYING SYSTEM MECHANISM:
To demonstrate system resilience, the form was submitted with empty required fields and negative numeric values (price=-10, quantity=-5). The controller intercepts the POST payload and executes rigorous validation rules before touching the database. An associative array $errors is populated with field-specific failure descriptions. Instead of crashing or losing user input, the controller re-renders products/create.php, retaining prior inputs via $old, highlighting invalid input borders in red, and injecting contextual error messages beneath each affected field.

KEY TECHNICAL COMPONENTS & FLOW:
- app/controllers/ProductController.php: store() validates SKU regex, SKU uniqueness, string lengths, and positive numbers.
- app/views/products/create.php: Conditional CSS class 'has-error' applies red accent borders and displays alert messages.
- Security Impact: Completely prevents malformed records or negative inventory levels from reaching the persistence tier.

Figure 5: Successful Product Registration & Session Flash Notification

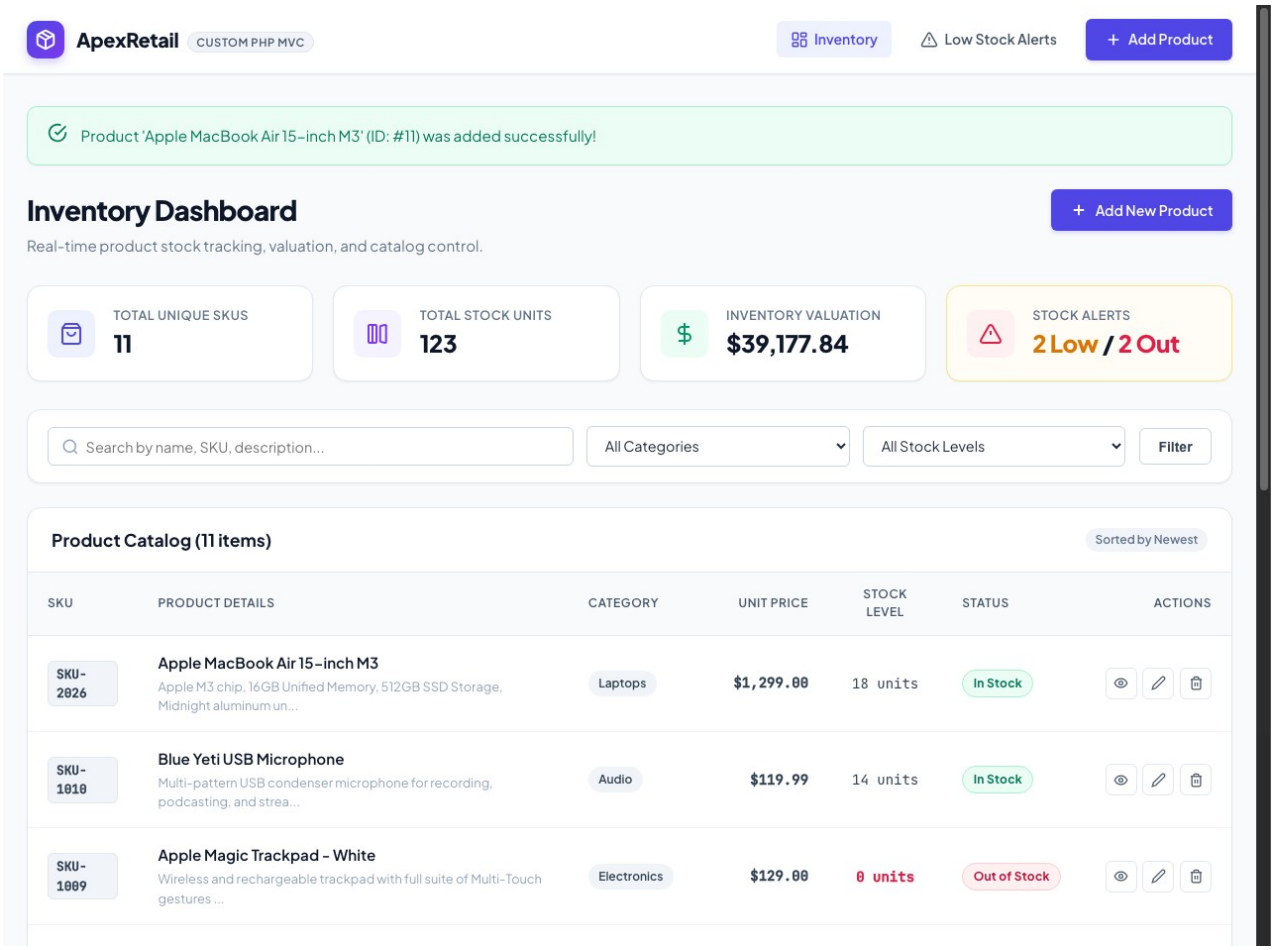


Figure 5 — Successful Product Registration & Session Flash Notification (ApexRetail Custom PHP MVC Engine)

Technical Breakdown & System Execution Details

ARCHITECTURAL LAYER: ProductController::store() -> Product::create() -> Session Flash -> HTTP 302 Redirect -> View

UNDERLYING SYSTEM MECHANISM:
A valid product was submitted: 'Apple MacBook Air 15-inch M3' (SKU-2026, Price: $1,299.00, Qty: 18, Category: Laptops). The controller passes sanitized inputs to Product::create(), which executes a PDO prepared INSERT query. Upon receiving the newly generated auto-increment ID (#11), the controller calls setFlash('success', 'Product ... was added successfully!') and issues an HTTP 302 redirect to /products. The front controller initializes the session, displays the green flash confirmation banner, clears the flash state, and renders the updated catalog. Total valuation automatically recalculated to $39,177.84.

KEY TECHNICAL COMPONENTS & FLOW:
- app/core/Model.php: execute() runs PDO::prepare() with named parameters (:sku, :name, :category, :price, :quantity).
- app/core/Controller.php: setFlash() writes notification to $_SESSION['flash']; getFlash() reads and unsets it.
- app/views/layouts/header.php: Automatically checks getFlash() and renders alert banner at top of viewport.

Figure 6: Comprehensive Product Specification & Stock Gauge Indicator

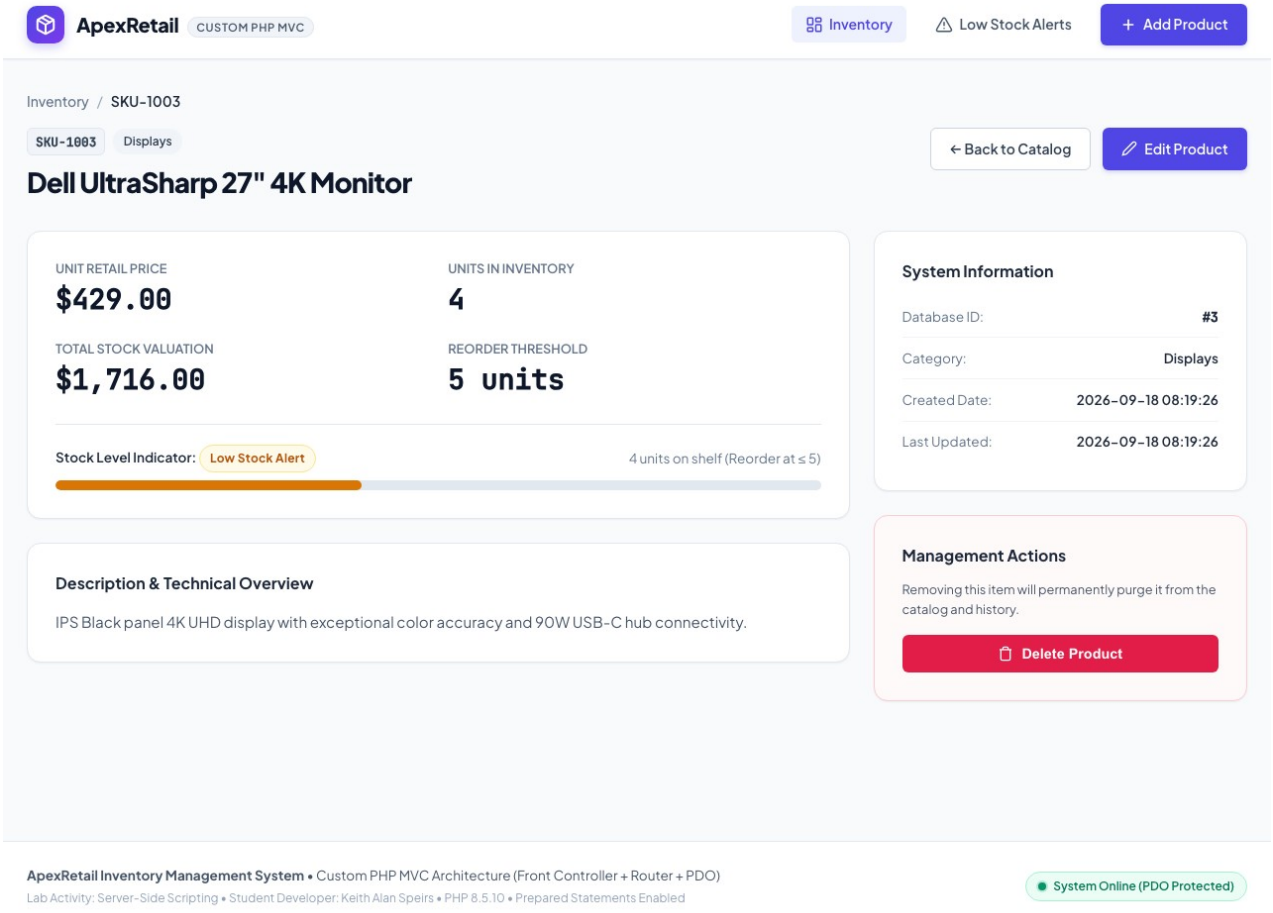


Figure 6 — Comprehensive Product Specification & Stock Gauge Indicator (ApexRetail Custom PHP MVC Engine)

Technical Breakdown & System Execution Details

ARCHITECTURAL LAYER: Router Dynamic Regex -> ProductController::show($id) -> Product::getById() -> View

UNDERLYING SYSTEM MECHANISM:
Navigating to /products/3 triggers regex pattern matching in Router.php: '#^/products/(?P<id>[^/]+)$#', passing '3' into ProductController::show($id). The controller queries the database via Product::getById(3). The view computes the item's individual stock valuation ($1,716.00) and calculates the stock health percentage (4 units on shelf vs 5 threshold), rendering an animated amber gauge indicator. System metadata (DB ID, timestamps) and management actions (Edit, Delete) are partitioned into an ergonomic sidebar.

KEY TECHNICAL COMPONENTS & FLOW:
- app/core/Router.php: Extracts numeric parameter from URI using preg_match.
- app/models/Product.php: getById() executes SELECT * FROM products WHERE id = :id LIMIT 1.
- app/views/products/show.php: Visual health gauge calculation and XSS-safe output encoding via htmlspecialchars().

Figure 7: Pre-Populated Product Modification Interface (Update Operation)

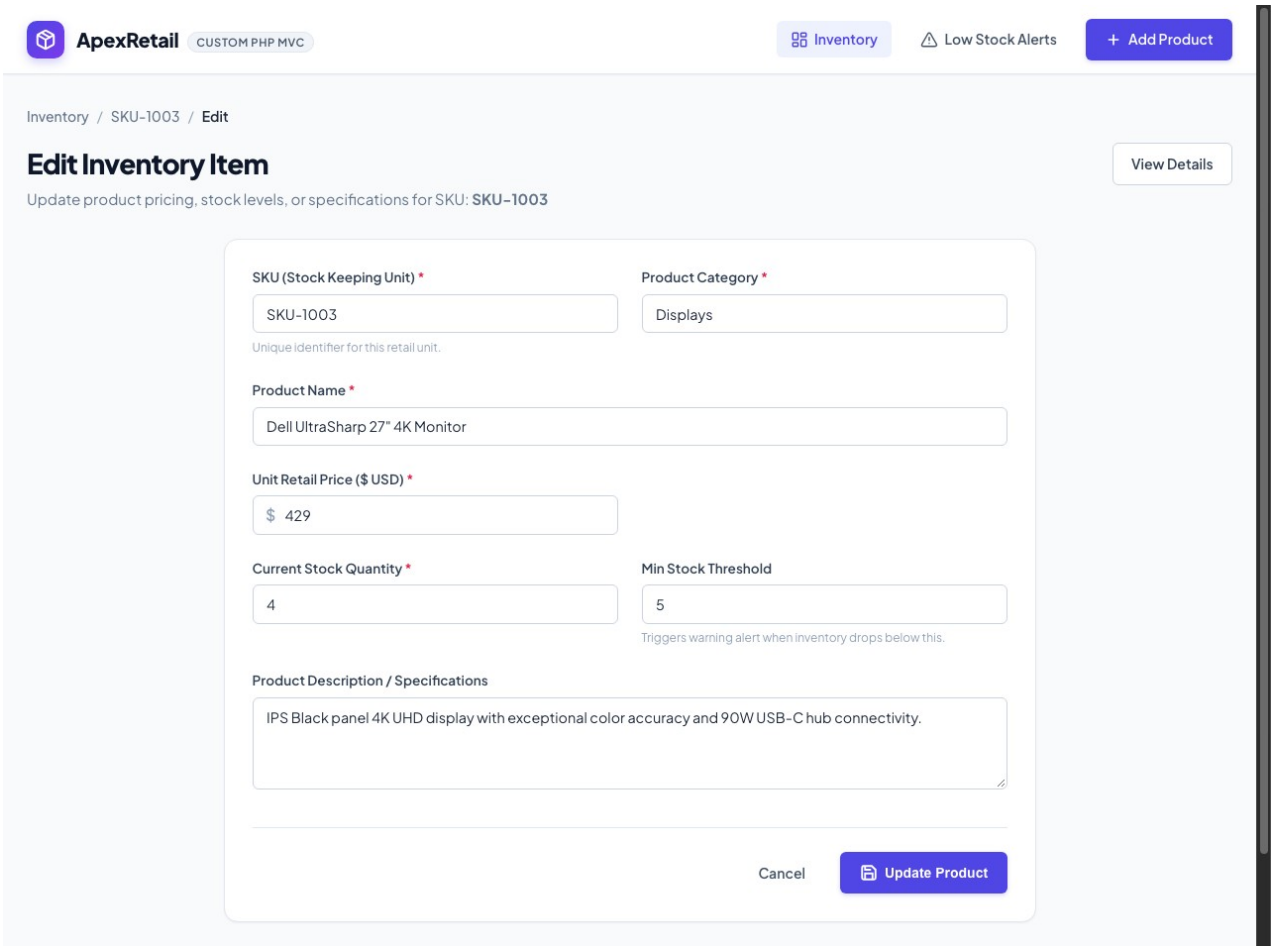


Figure 7 — Pre-Populated Product Modification Interface (Update Operation) (ApexRetail Custom PHP MVC Engine)

Technical Breakdown & System Execution Details

ARCHITECTURAL LAYER: Router -> ProductController::edit($id) -> Product::getById() -> View (products/edit.php)

UNDERLYING SYSTEM MECHANISM:
Clicking 'Edit Product' loads /products/3/edit. The controller fetches the current database record and injects it into products/edit.php. All input controls (SKU, Name, Category, Price, Quantity, Min Stock, Description) are bound to existing values. When updating, the controller ensures SKU uniqueness validation excludes the current item's ID so that unchanged SKUs do not trigger collision errors.

KEY TECHNICAL COMPONENTS & FLOW:
• app/controllers/ProductController.php: edit() verifies product existence before rendering.
• app/models/Product.php: getBySku($sku, $excludeId) uses 'SELECT * FROM products WHERE sku = :sku AND id != :exclude_id'.
• app/views/products/edit.php: Pre-fills input fields while allowing modification of any attribute.

Figure 8: Dynamic Stock Level Re-Evaluation & Alert Threshold Triggering

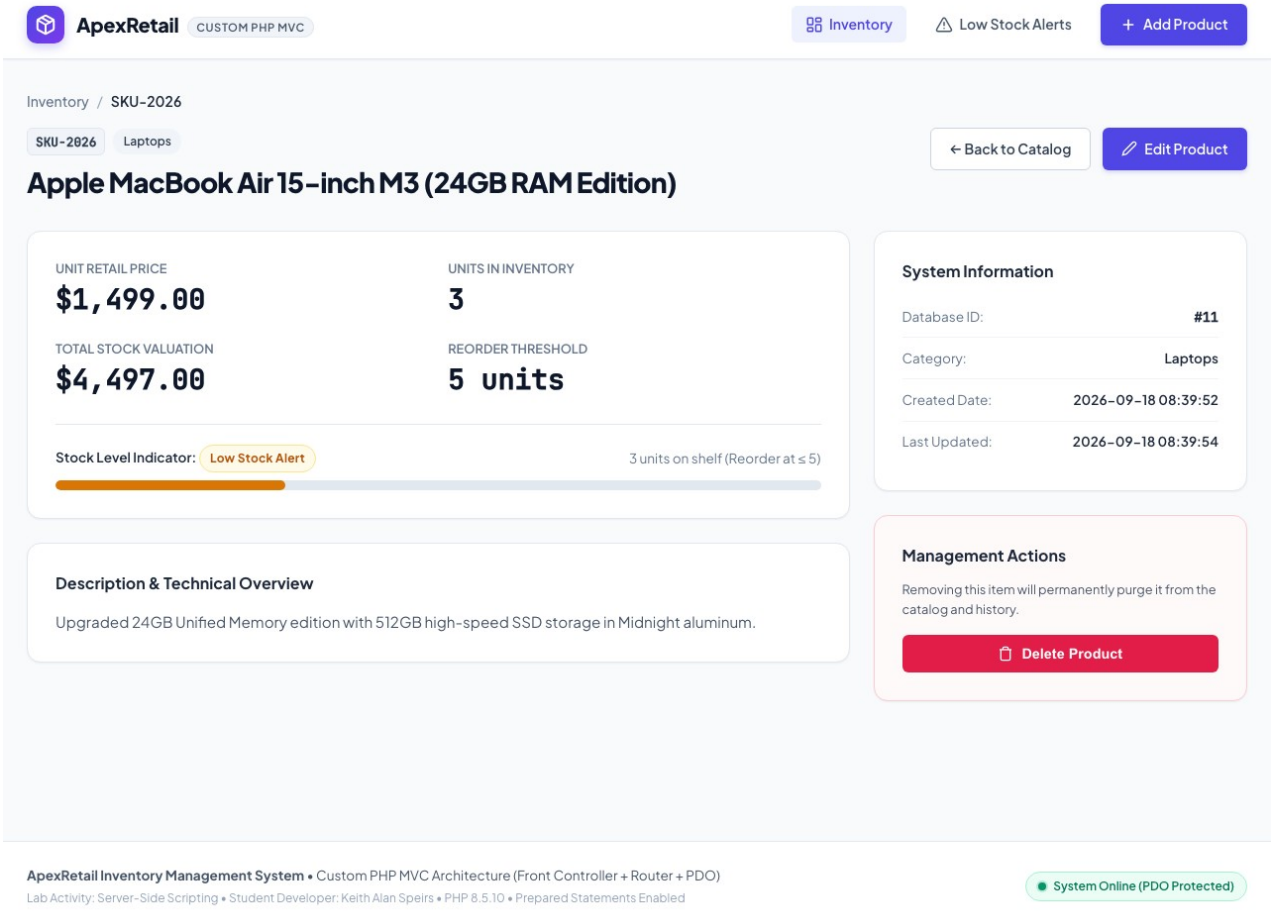


Figure 8 — Dynamic Stock Level Re-Evaluation & Alert Threshold Triggering (ApexRetail Custom PHP MVC Engine)

Technical Breakdown & System Execution Details

ARCHITECTURAL LAYER: ProductController::update($id) -> Product::update() -> View Re-render

UNDERLYING SYSTEM MECHANISM:
The created MacBook Air (#11) was modified: upgraded to '24GB RAM Edition', price increased to $1,499.00, and quantity decreased to 3 units (below the min_stock threshold of 5). The controller executes a PDO UPDATE query. Viewing the updated item demonstrates immediate re-evaluation: total valuation recalculates to $4,497.00, and the stock health indicator shifts from healthy green to amber 'Low Stock Alert' with an updated progress bar.

KEY TECHNICAL COMPONENTS & FLOW:
- app/models/Product.php: update() executes parameterized SQL UPDATE with CURRENT_TIMESTAMP.
- app/views/products/show.php: Automatically detects quantity <= min_stock and updates gauge styling to bg-amber.
- Real-Time Business Logic: Immediate visual alert without hardcoding statuses in database tables.

Figure 9: Secure Product Deletion & Automatic Catalog Refresh

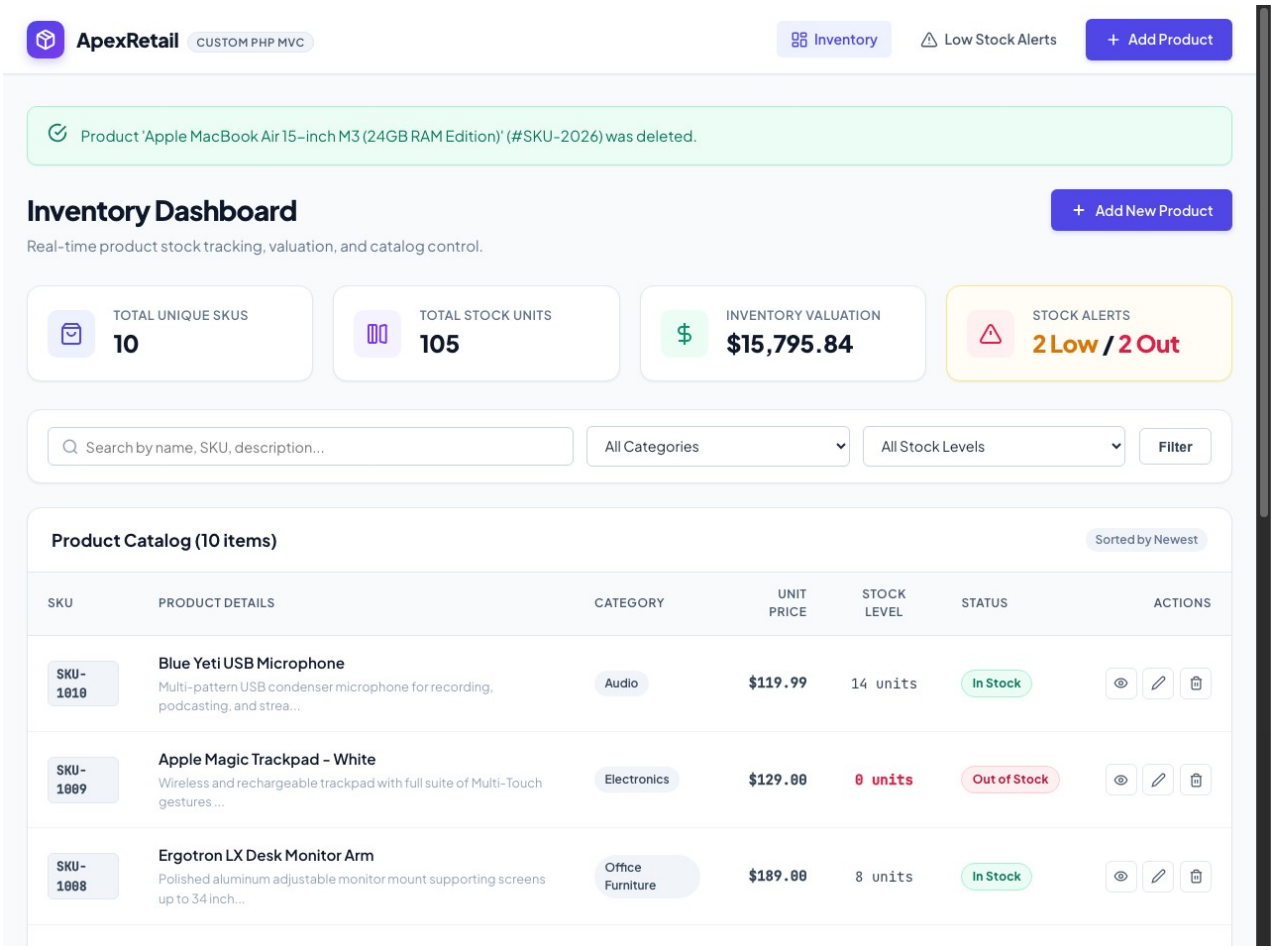


Figure 9 — Secure Product Deletion & Automatic Catalog Refresh (ApexRetail Custom PHP MVC Engine)

Technical Breakdown & System Execution Details

ARCHITECTURAL LAYER: JavaScript Confirmation -> POST /products/{id}/delete -> Product::delete() -> Flash Feedback

UNDERLYING SYSTEM MECHANISM:
When staff clicks the delete icon, an inline JavaScript confirmation dialog prompts: 'Are you sure you want to delete...?'. Upon confirmation, a POST request is dispatched to /products/11/delete. The controller calls Product::delete(11), executing 'DELETE FROM products WHERE id = :id'. The controller sets a flash notification 'Product ... was deleted', redirects to the dashboard, and displays the success banner. Total catalog count immediately decrements to 10 items.

KEY TECHNICAL COMPONENTS & FLOW:
- app/views/products/index.php: Secure POST form wrapping delete button with onsubmit confirmation guard.
- app/controllers/ProductController.php: delete() checks row count to ensure deletion occurred.
- app/core/Model.php: PDOStatement rowCount() verifies record removal before issuing redirect.

V. CORE SOURCE CODE IMPLEMENTATION

Below are the complete, unedited source code files implementing the custom PHP MVC architecture, showcasing clean separation of concerns, native autoloading, expressive routing, and secure PDO abstraction.

1. Front Controller (public/index.php)

Serves as the single point of entry for the application. Autoloads classes, starts the session, defines all CRUD routes, and dispatches requests:

```
// --- public/index.php ---
<?php
/**
 * Front Controller
 * Single entry point for all HTTP requests in the custom PHP MVC Framework
 */

declare(strict_types=1);

// Static file passthrough for PHP built-in development server
if (php_sapi_name() === 'cli-server') {
    $filePath = __DIR__ . parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);
    if (is_file($filePath)) {
        return false;
    }
}

// Initialize Session for Flash Alerts
if (session_status() === PHP_SESSION_NONE) {
    session_start();
}

// Set application root directory
define('ROOT_PATH', dirname(__DIR__));

// PSR-4 Style Autoloader for the "App" namespace
spl_autoload_register(function ($class) {
    $prefix = 'App\\';
    $baseDir = ROOT_PATH . '/app/';

    $len = strlen($prefix);
    if (strncmp($prefix, $class, $len) !== 0) {
        return;
    }

    $relativeClass = substr($class, $len);
    $file = $baseDir . str_replace('\\', '/', $relativeClass) . '.php';

    if (file_exists($file)) {
        require $file;
    }
});

// Import Core Classes
use App\Core\Router;
use App\Controllers\ProductController;

// Initialize Request Router
$router = new Router();

// Define Application Routes (CRUD Operations)
// Read: Dashboard and Catalog
$router->get('/', ProductController::class, 'index');
$router->get('/products', ProductController::class, 'index');

// Create: Add Product Form and Submission
$router->get('/products/create', ProductController::class, 'create');
$router->post('/products/create', ProductController::class, 'store');
```

```
// Read: Product Detail View
$router->get('/products/{id}', ProductController::class, 'show');

// Update: Edit Product Form and Submission
$router->get('/products/{id}/edit', ProductController::class, 'edit');
$router->post('/products/{id}/edit', ProductController::class, 'update');

// Delete: Remove Product Action
$router->post('/products/{id}/delete', ProductController::class, 'delete');
$router->get('/products/{id}/delete', ProductController::class, 'delete');

// Dispatch Request
$requestUri = $_SERVER['REQUEST_URI'] ?? '/';
$requestMethod = $_SERVER['REQUEST_METHOD'] ?? 'GET';

$router->dispatch($requestUri, $requestMethod);
```

2. Expressive Request Router (app/core/Router.php)

Parses the requested URI path, converts wildcard patterns (e.g. {id}) to named regular expression capture groups, and dispatches to controllers:

```
// --- app/core/Router.php ---
<?php
/**
 * Expressive Front Controller Request Router
 * Custom PHP MVC Framework
 */

namespace App\Core;

class Router
{
    private array $routes = [];

    /**
     * Register a GET route
     */
    public function get(string $pattern, string $controller, string $action): void
    {
        $this->addRoute('GET', $pattern, $controller, $action);
    }

    /**
     * Register a POST route
     */
    public function post(string $pattern, string $controller, string $action): void
    {
        $this->addRoute('POST', $pattern, $controller, $action);
    }

    /**
     * Store route specification
     */
    public function addRoute(string $method, string $pattern, string $controller, string $action):
void
    {
        $this->routes[] = [
            'method' => strtoupper($method),
```

```
            'pattern' => $pattern,
            'controller' => $controller,
            'action' => $action,
        ];
    }

    /**
     * Dispatch the current HTTP request to matching controller and action
     */
    public function dispatch(string $uri, string $method): void
    {
        // Clean and normalize the request URI
        $parsedUrl = parse_url($uri, PHP_URL_PATH);
        $path = '/' . trim($parsedUrl, '/');

        $method = strtoupper($method);
        if ($method === 'HEAD') {
            $method = 'GET';
        }

        foreach ($this->routes as $route) {
            if ($route['method'] !== $method) {
                continue;
            }

            // Convert route pattern {param} to regex named group
            $patternRegex = preg_replace('/\{([a-zA-Z0-9_]+)\}/', '(?P<$1>[^/]+)',
$route['pattern']);
            $patternRegex = '#^' . $patternRegex . '$#';

            if (preg_match($patternRegex, $path, $matches)) {
                $params = [];
                foreach ($matches as $key => $value) {
                    if (is_string($key)) {
                        $params[$key] = $value;
                    }
                }

                $controllerClass = $route['controller'];
                $actionName = $route['action'];

                if (!class_exists($controllerClass)) {
                    http_response_code(500);
                    die("Controller class '{$controllerClass}' not found.");
                }

                $controllerInstance = new $controllerClass();

                if (!method_exists($controllerInstance, $actionName)) {
                    http_response_code(500);
                    die("Action '{$actionName}' not found on controller '{$controllerClass}'.");
                }

                // Call the controller action with parameters
                call_user_func_array([$controllerInstance, $actionName], $params);
                return;
            }
        }

        // 404 Not Found Handler
        http_response_code(404);
        echo "<!DOCTYPE html><html><head><title>404 Not Found</title><style>body{font-family:sans-
serif;padding:50px;text-align:center;background:#f8fafc;color:#1e293b}h1{font-size:48px;margin-
```

```
bottom:10px}p{color:#64748b}a{color:#4f46e5;text-decoration:none;font-weight:600}</style></
head><body><h1>404</h1><p>The requested page <code>" . htmlspecialchars($path) . "</code> was not
found.</p><p><a href='/'>&larr; Return to Inventory Dashboard</a></p></body></html>";
    }
}
```

3. Database Singleton Connection (app/core/Database.php)

Implements the Singleton design pattern to maintain a single PDO instance. Configures strict error modes and prepared statement emulation flags:

```
// --- app/core/Database.php ---
<?php
/**
 * Database Singleton Connection Handler
 * Custom PHP MVC Framework
 */

namespace App\Core;

use PDO;
use PDOException;

class Database
{
    private static ?PDO $instance = null;

    /**
     * Private constructor to prevent direct instantiation
     */
    private function __construct()
    {
    }

    /**
     * Get or initialize the PDO instance
     */
    public static function getConnection(): PDO
    {
        if (self::$instance === null) {
            $config = require dirname(__DIR__) . '/config/database.php';
            $driver = $config['driver'] ?? 'sqlite';

            try {
                if ($driver === 'sqlite') {
                    $dbPath = $config['sqlite']['path'];
                    $dir = dirname($dbPath);
                    if (!is_dir($dir)) {
                        mkdir($dir, 0777, true);
                    }

                    $isNew = !file_exists($dbPath) || filesize($dbPath) === 0;
                    self::$instance = new PDO("sqlite:" . $dbPath);
                    self::$instance->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
                    self::$instance->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE, PDO::FETCH_ASSOC);

                    if ($isNew) {
                        self::initializeSqliteSchema(self::$instance);
                    }
                } elseif ($driver === 'mysql') {
```

```
                    $mysql = $config['mysql'];
                    $dsn = sprintf(
                        'mysql:host=%s;port=%s;dbname=%s;charset=%s',
                        $mysql['host'],
                        $mysql['port'],
                        $mysql['database'],
                        $mysql['charset']
                    );

                    self::$instance = new PDO(
                        $dsn,
                        $mysql['username'],
                        $mysql['password'],
                        [
                            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                            PDO::ATTR_EMULATE_PREPARES => false,
                        ]
                    );
                } else {
                    throw new PDOException("Unsupported database driver: {$driver}");
                }
            } catch (PDOException $e) {
                die("Database Connection Error: " . htmlspecialchars($e->getMessage()));
            }
        }

        return self::$instance;
    }

    /**
     * Initialize SQLite tables and seed data if database is fresh
     */
    private static function initializeSqliteSchema(PDO $pdo): void
    {
        $sql = "
        CREATE TABLE IF NOT EXISTS products (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            sku TEXT NOT NULL UNIQUE,
            name TEXT NOT NULL,
            category TEXT NOT NULL,
            price REAL NOT NULL,
            quantity INTEGER NOT NULL DEFAULT 0,
            min_stock INTEGER NOT NULL DEFAULT 5,
            description TEXT,
            created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
            updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
        );

        INSERT INTO products (sku, name, category, price, quantity, min_stock, description) VALUES
        ('SKU-1001', 'Logitech MX Master 3S Wireless Mouse', 'Electronics', 99.99, 28, 8, 'Ergonomic
performance mouse with ultra-fast MagSpeed scrolling and 8K DPI tracking.'),
        ('SKU-1002', 'Keychron K2 Mechanical Keyboard', 'Electronics', 79.50, 15, 6, 'Compact 75%
layout wireless mechanical keyboard with Gateron brown switches and RGB backlighting.'),
        ('SKU-1003', 'Dell UltraSharp 27\" 4K Monitor', 'Displays', 429.00, 4, 5, 'IPS Black panel 4K
UHD display with exceptional color accuracy and 90W USB-C hub connectivity.'),
        ('SKU-1004', 'Sony WH-1000XM5 Noise Canceling Headphones', 'Audio', 348.00, 12, 5, 'Industry-
leading noise canceling over-ear headphones with 30-hour battery life.'),
        ('SKU-1005', 'Anker 737 Power Bank (PowerCore 24K)', 'Accessories', 149.99, 2, 6, '24000mAh
3-port portable charger with smart digital display and 140W max output.'),
        ('SKU-1006', 'CalDigit TS4 Thunderbolt 4 Dock', 'Accessories', 399.95, 0, 4, '18-port
Thunderbolt 4 dock providing 98W host charging and dual 6K display support.'),
        ('SKU-1007', 'SanDisk 1TB Extreme Portable SSD', 'Storage', 109.99, 22, 10, 'Rugged NVMe
```

```
solid state drive with up to 1050MB/s read speeds and IP55 water/dust resistance.'),
        ('SKU-1008', 'Ergotron LX Desk Monitor Arm', 'Office Furniture', 189.00, 8, 4, 'Polished
aluminum adjustable monitor mount supporting screens up to 34 inches.'),
        ('SKU-1009', 'Apple Magic Trackpad - White', 'Electronics', 129.00, 0, 5, 'Wireless and
rechargeable trackpad with full suite of Multi-Touch gestures and Force Touch.'),
        ('SKU-1010', 'Blue Yeti USB Microphone', 'Audio', 119.99, 14, 5, 'Multi-pattern USB condenser
microphone for recording, podcasting, and streaming.');
        ";

        $pdo->exec($sql);
    }
}
```

4. Base Model Class (app/core/Model.php)

Abstract base class wrapping PDO execution, providing parameterized query helpers fetchAll() and fetchOne():

```
// --- app/core/Model.php ---
<?php
/**
 * Base Model Class
 * Custom PHP MVC Framework
 */

namespace App\Core;

use PDO;
use PDOStatement;

abstract class Model
{
    protected PDO $db;

    public function __construct()
    {
        $this->db = Database::getConnection();
    }

    /**
     * Prepare and execute a parameterized SQL query
     */
    protected function execute(string $sql, array $params = []): PDOStatement
    {
        $stmt = $this->db->prepare($sql);
        $stmt->execute($params);
        return $stmt;
    }

    /**
     * Fetch all matching records as an associative array
     */
    protected function fetchAll(string $sql, array $params = []): array
    {
        return $this->execute($sql, $params)->fetchAll();
    }

    /**
     * Fetch a single matching record or false
     */
    protected function fetchOne(string $sql, array $params = []): ?array
```

```
    {
        $record = $this->execute($sql, $params)->fetch();
        return $record ?: null;
    }

    /**
     * Get the ID of the last inserted record
     */
    protected function lastInsertId(): string
    {
        return $this->db->lastInsertId();
    }
}
```

5. Base Controller Class (app/core/Controller.php)

Abstract base controller providing view rendering with layout nesting, session flash messaging, and request input sanitization:

```
// --- app/core/Controller.php ---
<?php
/**
 * Base Controller Class
 * Custom PHP MVC Framework
 */

namespace App\Core;

abstract class Controller
{
    /**
     * Render a view within standard layout templates
     */
    protected function render(string $viewPath, array $data = [], bool $useLayout = true): void
    {
        // Extract variables for the view
        extract($data);

        // Flash message handling
        $flash = $this->getFlash();

        $baseViewsDir = dirname(__DIR__) . '/views/';
        $viewFile = $baseViewsDir . $viewPath . '.php';

        if (!file_exists($viewFile)) {
            die("View template not found: " . htmlspecialchars($viewFile));
        }

        if ($useLayout) {
            $headerFile = $baseViewsDir . 'layouts/header.php';
            $footerFile = $baseViewsDir . 'layouts/footer.php';

            if (file_exists($headerFile)) {
                require $headerFile;
            }

            require $viewFile;

            if (file_exists($footerFile)) {
                require $footerFile;
```

```
            }
        } else {
            require $viewFile;
        }
    }

    /**
     * Redirect to a specific URL path
     */
    protected function redirect(string $path): void
    {
        header("Location: " . $path);
        exit;
    }

    /**
     * Set a one-time flash notification message
     */
    protected function setFlash(string $type, string $message): void
    {
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }
        $_SESSION['flash'] = [
            'type' => $type, // 'success', 'danger', 'warning', 'info'
            'message' => $message,
        ];
    }

    /**
     * Retrieve and clear the flash notification message
     */
    protected function getFlash(): ?array
    {
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }
        if (isset($_SESSION['flash'])) {
            $flash = $_SESSION['flash'];
            unset($_SESSION['flash']);
            return $flash;
        }
        return null;
    }

    /**
     * Helper to retrieve sanitized POST inputs
     */
    protected function post(string $key, $default = null)
    {
        return isset($_POST[$key]) ? trim($_POST[$key]) : $default;
    }

    /**
     * Helper to retrieve sanitized GET inputs
     */
    protected function query(string $key, $default = null)
    {
        return isset($_GET[$key]) ? trim($_GET[$key]) : $default;
    }
}
```

6. Product Entity Model (app/models/Product.php)

Encapsulates all SQL logic, CRUD operations, statistical aggregations, and SKU uniqueness validation using prepared statements:

```
// --- app/models/Product.php ---
<?php
/**
 * Product Model
 * Handles all database interactions and business logic for Inventory Items
 */

namespace App\Models;

use App\Core\Model;
use PDO;

class Product extends Model
{
    /**
     * Retrieve products with optional filtering and search
     */
    public function getAll(?string $search = null, ?string $category = null, ?string $stockStatus =
null): array
    {
        $sql = "SELECT * FROM products WHERE 1=1";
        $params = [];

        if (!empty($search)) {
            $sql .= " AND (name LIKE :search OR sku LIKE :search_sku OR description
LIKE :search_desc)";
            $params[':search'] = '%' . $search . '%';
            $params[':search_sku'] = '%' . $search . '%';
            $params[':search_desc'] = '%' . $search . '%';
        }

        if (!empty($category)) {
            $sql .= " AND category = :category";
            $params[':category'] = $category;
        }

        if ($stockStatus === 'low') {
            $sql .= " AND quantity > 0 AND quantity <= min_stock";
        } elseif ($stockStatus === 'out') {
            $sql .= " AND quantity = 0";
        } elseif ($stockStatus === 'in') {
            $sql .= " AND quantity > min_stock";
        }

        $sql .= " ORDER BY id DESC";

        return $this->fetchAll($sql, $params);
    }

    /**
     * Retrieve a single product by ID
     */
    public function getById(int $id): ?array
    {
        $sql = "SELECT * FROM products WHERE id = :id LIMIT 1";
        return $this->fetchOne($sql, [':id' => $id]);
    }
```

```
    /**
     * Check if a SKU exists, optionally excluding a specific product ID (for updates)
     */
    public function getBySku(string $sku, ?int $excludeId = null): ?array
    {
        if ($excludeId !== null) {
            $sql = "SELECT * FROM products WHERE sku = :sku AND id != :exclude_id LIMIT 1";
            return $this->fetchOne($sql, [':sku' => $sku, ':exclude_id' => $excludeId]);
        }

        $sql = "SELECT * FROM products WHERE sku = :sku LIMIT 1";
        return $this->fetchOne($sql, [':sku' => $sku]);
    }

    /**
     * Insert a new inventory product
     */
    public function create(array $data): int
    {
        $sql = "INSERT INTO products (sku, name, category, price, quantity, min_stock, description,
created_at, updated_at)
                VALUES (:sku, :name, :category, :price, :quantity, :min_stock, :description,
CURRENT_TIMESTAMP, CURRENT_TIMESTAMP)";

        $params = [
            ':sku' => $data['sku'],
            ':name' => $data['name'],
            ':category' => $data['category'],
            ':price' => $data['price'],
            ':quantity' => $data['quantity'],
            ':min_stock' => $data['min_stock'] ?? 5,
            ':description' => $data['description'] ?? null,
        ];

        $this->execute($sql, $params);
        return (int)$this->lastInsertId();
    }

    /**
     * Update an existing product
     */
    public function update(int $id, array $data): bool
    {
        $sql = "UPDATE products
                SET sku = :sku,
                    name = :name,
                    category = :category,
                    price = :price,
                    quantity = :quantity,
                    min_stock = :min_stock,
                    description = :description,
                    updated_at = CURRENT_TIMESTAMP
                WHERE id = :id";

        $params = [
            ':id' => $id,
            ':sku' => $data['sku'],
            ':name' => $data['name'],
            ':category' => $data['category'],
            ':price' => $data['price'],
            ':quantity' => $data['quantity'],
            ':min_stock' => $data['min_stock'] ?? 5,
            ':description' => $data['description'] ?? null,
```

```
        ];

        $stmt = $this->execute($sql, $params);
        return $stmt->rowCount() > 0;
    }

    /**
     * Delete a product by ID
     */
    public function delete(int $id): bool
    {
        $sql = "DELETE FROM products WHERE id = :id";
        $stmt = $this->execute($sql, [':id' => $id]);
        return $stmt->rowCount() > 0;
    }

    /**
     * Compute dashboard statistics: total items, total inventory value, low stock, out of stock
     */
    public function getStatistics(): array
    {
        $sql = "SELECT
                    COUNT(*) as total_items,
                    COALESCE(SUM(quantity), 0) as total_units,
                    COALESCE(SUM(price * quantity), 0) as total_value,
                    SUM(CASE WHEN quantity > 0 AND quantity <= min_stock THEN 1 ELSE 0 END) as
low_stock_count,
                    SUM(CASE WHEN quantity = 0 THEN 1 ELSE 0 END) as out_of_stock_count
                FROM products";

        $stats = $this->fetchOne($sql);
        return $stats ?: [
            'total_items' => 0,
            'total_units' => 0,
            'total_value' => 0.0,
            'low_stock_count' => 0,
            'out_of_stock_count' => 0,
        ];
    }

    /**
     * Get distinct product categories
     */
    public function getCategories(): array
    {
        $sql = "SELECT DISTINCT category FROM products ORDER BY category ASC";
        $rows = $this->fetchAll($sql);
        return array_column($rows, 'category');
    }
}
```

7. Product Request Controller (app/controllers/ProductController.php)

Orchestrates user actions, performs field validation, manages flash feedback, and routes execution:

```
// --- app/controllers/ProductController.php ---
<?php
/**
 * Product Controller
 * Orchestrates request handling, business logic, validation, and view rendering
```

```
 */

namespace App\Controllers;

use App\Core\Controller;
use App\Models\Product;

class ProductController extends Controller
{
    private Product $productModel;

    public function __construct()
    {
        $this->productModel = new Product();
    }

    /**
     * Display the inventory dashboard & product listing
     */
    public function index(): void
    {
        $search = $this->query('search', '');
        $category = $this->query('category', '');
        $stockStatus = $this->query('stock_status', '');

        $products = $this->productModel->getAll($search, $category, $stockStatus);
        $stats = $this->productModel->getStatistics();
        $categories = $this->productModel->getCategories();

        $this->render('products/index', [
            'pageTitle' => 'Inventory Dashboard',
            'products' => $products,
            'stats' => $stats,
            'categories' => $categories,
            'search' => $search,
            'currentCategory' => $category,
            'stockStatus' => $stockStatus,
        ]);
    }

    /**
     * Display a specific product details view
     */
    public function show(string $id): void
    {
        $product = $this->productModel->getById((int)$id);

        if (!$product) {
            $this->setFlash('danger', 'The requested product was not found.');
            $this->redirect('/products');
        }

        $this->render('products/show', [
            'pageTitle' => 'Product Details - ' . $product['name'],
            'product' => $product,
        ]);
    }

    /**
     * Show the form for creating a new product
     */
    public function create(): void
    {
```

```
        $categories = $this->productModel->getCategories();
        if (empty($categories)) {
            $categories = ['Electronics', 'Accessories', 'Audio', 'Displays', 'Storage', 'Office
Furniture'];
        }

        $this->render('products/create', [
            'pageTitle' => 'Add New Product',
            'categories' => $categories,
            'errors' => [],
            'old' => [],
        ]);
    }

    /**
     * Store a newly created product in the database
     */
    public function store(): void
    {
        $sku = strtoupper(trim($this->post('sku', '')));
        $name = trim($this->post('name', ''));
        $category = trim($this->post('category', ''));
        $price = trim($this->post('price', ''));
        $quantity = trim($this->post('quantity', ''));
        $minStock = trim($this->post('min_stock', '5'));
        $description = trim($this->post('description', ''));

        $errors = [];

        // Validation Rules
        if (empty($sku)) {
            $errors['sku'] = 'SKU (Stock Keeping Unit) is required.';
        } elseif (!preg_match('/^[A-Z0-9_-]+$/i', $sku)) {
            $errors['sku'] = 'SKU must only contain alphanumeric characters, hyphens, and
underscores.';
        } elseif ($this->productModel->getBySku($sku)) {
            $errors['sku'] = 'A product with this SKU already exists.';
        }

        if (empty($name)) {
            $errors['name'] = 'Product name is required.';
        } elseif (strlen($name) < 3) {
            $errors['name'] = 'Product name must be at least 3 characters long.';
        }

        if (empty($category)) {
            $errors['category'] = 'Please select or enter a category.';
        }

        if ($price === '' || !is_numeric($price) || (float)$price < 0) {
            $errors['price'] = 'Please enter a valid non-negative unit price.';
        }

        if ($quantity === '' || !is_numeric($quantity) || (int)$quantity < 0) {
            $errors['quantity'] = 'Please enter a valid non-negative quantity.';
        }

        if ($minStock !== '' && (!is_numeric($minStock) || (int)$minStock < 0)) {
            $errors['min_stock'] = 'Minimum stock threshold must be a non-negative integer.';
        }

        $oldData = [
            'sku' => $sku,
```

```
            'name' => $name,
            'category' => $category,
            'price' => $price,
            'quantity' => $quantity,
            'min_stock' => $minStock,
            'description' => $description,
        ];

        if (!empty($errors)) {
            $categories = $this->productModel->getCategories();
            $this->render('products/create', [
                'pageTitle' => 'Add New Product',
                'categories' => $categories,
                'errors' => $errors,
                'old' => $oldData,
            ]);
            return;
        }

        $productId = $this->productModel->create([
            'sku' => $sku,
            'name' => $name,
            'category' => $category,
            'price' => (float)$price,
            'quantity' => (int)$quantity,
            'min_stock' => (int)$minStock,
            'description' => $description,
        ]);

        $this->setFlash('success', "Product '{$name}' (ID: #{$productId}) was added successfully!");
        $this->redirect('/products');
    }

    /**
     * Show the form for editing an existing product
     */
    public function edit(string $id): void
    {
        $product = $this->productModel->getById((int)$id);

        if (!$product) {
            $this->setFlash('danger', 'Product not found.');
            $this->redirect('/products');
        }

        $categories = $this->productModel->getCategories();

        $this->render('products/edit', [
            'pageTitle' => 'Edit Product - ' . $product['name'],
            'product' => $product,
            'categories' => $categories,
            'errors' => [],
            'old' => $product,
        ]);
    }

    /**
     * Update an existing product in the database
     */
    public function update(string $id): void
    {
        $productId = (int)$id;
        $product = $this->productModel->getById($productId);
```

```
        if (!$product) {
            $this->setFlash('danger', 'Product not found.');
            $this->redirect('/products');
        }

        $sku = strtoupper(trim($this->post('sku', '')));
        $name = trim($this->post('name', ''));
        $category = trim($this->post('category', ''));
        $price = trim($this->post('price', ''));
        $quantity = trim($this->post('quantity', ''));
        $minStock = trim($this->post('min_stock', '5'));
        $description = trim($this->post('description', ''));

        $errors = [];

        if (empty($sku)) {
            $errors['sku'] = 'SKU is required.';
        } elseif (!preg_match('/^[A-Z0-9_-]+$/i', $sku)) {
            $errors['sku'] = 'SKU must only contain alphanumeric characters, hyphens, and
underscores.';
        } elseif ($this->productModel->getBySku($sku, $productId)) {
            $errors['sku'] = 'Another product with this SKU already exists.';
        }

        if (empty($name)) {
            $errors['name'] = 'Product name is required.';
        } elseif (strlen($name) < 3) {
            $errors['name'] = 'Product name must be at least 3 characters long.';
        }

        if (empty($category)) {
            $errors['category'] = 'Please select or enter a category.';
        }

        if ($price === '' || !is_numeric($price) || (float)$price < 0) {
            $errors['price'] = 'Please enter a valid non-negative unit price.';
        }

        if ($quantity === '' || !is_numeric($quantity) || (int)$quantity < 0) {
            $errors['quantity'] = 'Please enter a valid non-negative quantity.';
        }

        if ($minStock !== '' && (!is_numeric($minStock) || (int)$minStock < 0)) {
            $errors['min_stock'] = 'Minimum stock threshold must be a non-negative integer.';
        }

        $oldData = [
            'id' => $productId,
            'sku' => $sku,
            'name' => $name,
            'category' => $category,
            'price' => $price,
            'quantity' => $quantity,
            'min_stock' => $minStock,
            'description' => $description,
        ];

        if (!empty($errors)) {
            $categories = $this->productModel->getCategories();
            $this->render('products/edit', [
                'pageTitle' => 'Edit Product - ' . $product['name'],
                'product' => $product,
```

```
                'categories' => $categories,
                'errors' => $errors,
                'old' => $oldData,
            ]);
            return;
        }

        $this->productModel->update($productId, [
            'sku' => $sku,
            'name' => $name,
            'category' => $category,
            'price' => (float)$price,
            'quantity' => (int)$quantity,
            'min_stock' => (int)$minStock,
            'description' => $description,
        ]);

        $this->setFlash('success', "Product '{$name}' was updated successfully!");
        $this->redirect('/products');
    }

    /**
     * Delete a product
     */
    public function delete(string $id): void
    {
        $productId = (int)$id;
        $product = $this->productModel->getById($productId);

        if ($product) {
            $this->productModel->delete($productId);
            $this->setFlash('success', "Product '{$product['name']}' (#{$product['sku']}) was
deleted.");
        } else {
            $this->setFlash('danger', 'Product not found or already deleted.');
        }

        $this->redirect('/products');
    }
}
```

VI. SECURITY ANALYSIS & ARCHITECTURAL BEST PRACTICES

The implementation rigorously incorporates industry-standard defensive programming practices across all tiers:

1. Prevention of SQL Injection (SQLi)

SQL Injection occurs when untrusted user input is directly concatenated into SQL query strings. In this custom MVC framework, direct string interpolation into SQL is strictly forbidden. Every database query in Product.php uses PDO prepared statements:

```
// --- PDO Prepared Statement Defense Mechanism ---
// VULNERABLE APPROACH (DO NOT USE):
// $db->query("SELECT * FROM products WHERE sku = '" . $_POST['sku'] . "'");
// An attacker submitting "' OR '1'='1" bypasses authentication or extracts confidential data.

// SECURE CUSTOM MVC IMPLEMENTATION (USED IN PROJECT):
$sql = "SELECT * FROM products WHERE sku = :sku AND id != :exclude_id LIMIT 1";
```

```
$stmt = $this->db->prepare($sql);
$stmt->execute([':sku' => $sku, ':exclude_id' => $excludeId]);
return $stmt->fetch();
```

By setting PDO::ATTR_EMULATE_PREPARES => false in Database.php, PHP delegates query parsing directly to the database engine. The query template is compiled first, and parameter values are transmitted separately over the binary protocol. It is mathematically impossible for user input to alter the syntax tree of the compiled SQL query.

2. Cross-Site Scripting (XSS) Mitigation

All dynamic variables rendered inside HTML view templates pass through htmlspecialchars($var, ENT_QUOTES, 'UTF-8'). This converts special characters (<, >, ", ', &) into their respective HTML entities, neutralizing malicious JavaScript injection attempts from product descriptions or names.

3. Single Entry Point & Web Root Isolation

Only the public/ directory is exposed to the web server. The sensitive application core, models, controllers, configurations, and database files reside in parent directories above the document root. Direct HTTP requests to sensitive PHP classes or database files return 404/403, preventing source code exposure.

VII. STUDENT REFLECTION & CONCLUSION

> **Student Reflection — Keith Alan Speirs (BSIT)**
> Completing this laboratory activity provided profound insights into the foundational mechanics of web frameworks. Prior to building this custom MVC structure, concepts like routing, front controllers, and ORM query builders felt like 'magic' handled automatically by Laravel. Writing the router using regular expression matching and parameter extraction demystified how clean RESTful URLs (/products/{id}/edit) map to controller methods.
>
> Implementing the PDO database singleton reinforced the critical necessity of prepared statements. Seeing how easy it is to safeguard an application against SQL injection using parameter binding proved that security does not require heavyweight dependencies. Furthermore, structuring the code into distinct Models, Views, and Controllers made debugging intuitive: database errors were isolated to Product.php, form validation issues to ProductController.php, and layout issues to the view templates.
>
> In conclusion, this project demonstrated that a lightweight, custom PHP MVC framework can deliver enterprise-grade performance, clean maintainability, and robust security while maintaining a tiny resource footprint. The skills acquired in this lab will directly translate into building scalable, secure web architectures throughout my professional career.